Web development 1 - Verslag Labo 15

Bachelor in de **toegepaste informatica**

campus **Kortrijk**

Thomas De Groote

docent: **Pim Debaere**

academiejaar **2025-2026**

Opdracht 1: Paragrafen

```
const setup = () => {
  let opvallends = document.getElementsByClassName("belangrijk");
  for (let i = 0; i < opvallends.length; i++) {
    opvallends[i].classList.add("opvallend");
  }
};
window.addEventListener("load", setup);
```

Eerst wordt een setup functie gemaakt, in die setup functie wordt een array gemaakt met alle elementen met klasse 'belangrijk'.

Daaronder wordt een loop gemaakt, en voor elk object van de klasselijst wordt de klasse opvallend toegevoegd.

De addEventListener zorgt ervoor dat de setup functie meteen wordt uitgevoerd bij het opstarten van de pagina.

Opvragen van elementen

Naast getElementsByClassName zijn er ook nog andere accessors in JavaScript.

1. **document.getElementById("id")**
   Zoekt één element met een specifieke ID.
   Een ID mag maar één keer voorkomen in je HTML.

2. **document.getElementsByClassName("class")**
   Zoekt alle elementen met een bepaalde class.
   Je krijgt meerdere elementen terug.

3. **document.getElementsByTagName("tag")**
   Zoekt alle elementen met een bepaalde HTML-tag, zoals "div" of "p".

4. **document.getElementsByName("name")**
   Zoekt elementen met een bepaalde name-waarde.
   Wordt vaak gebruikt bij formulieren.

5. **document.querySelector("selector")**
   Zoekt het eerste element dat past bij een CSS-selector.
   Bijvoorbeeld: "#id" of ".class".

6. **document.querySelectorAll("selector")**
   Zoekt alle elementen die passen bij een CSS-selector.
   Dit is een moderne en veelgebruikte methode.

7. **element.querySelector("selector")**
   Zoekt binnen één specifiek element in plaats van de hele pagina.

8. **element.querySelectorAll("selector")**
   Zoekt meerdere elementen binnen één specifiek element.

Opdracht 2: Demonstratie slider

De aanpassing van de slider wordt telkens naar de console geprint.

Als de slider wordt losgelaten dan wordt de laatste output nogmaals naar de console geprint.

```
sliders[0].addEventListener("change", update);
sliders[0].addEventListener("input", update);
```

Hier wordt de EventListener aan de slider gekoppeld.

Bij verandering wordt de update functie uitgevoerd, bij input (= loslaten van de slider), wordt update nogmaals uitgevoerd, vandaar dat de laatste waarde 2x wordt afgedrukt.

```
colorDemos[0].style.backgroundColor="red";
```

In de setup functie wordt ook de rode background kleur gegeven aan alle elementen met de klasse colorDemo.

We schrijven sliders[0] en colorDemos[0] omdat we een element uit een array moeten halen, anders zouden we de hele array bedoelen en dat zou niet werken.

Opdracht 3: Colorpicker

```
let rood, green, blue;

const setup = () => {
    red = document.getElementById("red");
    green = document.getElementById("green");
    blue = document.getElementById("blue");

    red.addEventListener("input", update);
    green.addEventListener("input", update);
    blue.addEventListener("input", update);
}

const update = () => {
    let vierkant = document.getElementById("kleurblok");
    vierkant.style.backgroundColor = `rgb(${rood.value}, ${green.value}, ${blue.value})`;
}

window.addEventListener("load", setup);
```

JavaScript code vind je hierboven. Eerst worden de variabelen van de 3-slider elementen bovenaan geïnitialiseerd, zodat ze over het hele script gebruikt kunnen worden gebruikt.

In de setup functie worden de 3 sliders opgevraagd als element, daarna wordt er een EventListener op de sliders-elementen gezet om input waar te nemen (= live aanpassen). Als er input wordt waargenomen, wordt de functie update uitgevoerd.

In de functie update wordt het div element met ID vierkant opgevraagd en de lijn daaronder wordt de background kleur aangepast naar de values van de sliders.

Opdracht 4: Kleurenverwisselaar

```
let b1, b2, b3;

const setup = () => {
    let buttons = document.getElementsByTagName("button")

    buttons[0].addEventListener("click", veranderKleur);
    buttons[1].addEventListener("click", veranderKleur);
    buttons[2].addEventListener("click", veranderKleur);
}

const veranderKleur = (event) => {
    let button = event.target;
    let huidigeKleur = window.getComputedStyle(button).backgroundColor;

    if (huidigeKleur === "white" {
        button.style.backgroundColor = "blue";
    } else {
        button.style.backgroundColor = "white";
    }
}

window.addEventListener("load", setup);
```

Op de klik op 1 van de buttons wordt de functie ‘veranderKleur’ opgeroepen. De veranderKleur-functie kijkt vanaf welke knop de eventListener is gedetecteerd en vraagt dan de background color op. Als de background color ‘white’ is, wordt de achtergrondkleur op ‘blue’ gezet en omgekeerd.

```javascript
const setup = () => {
  document
    .getElementById("calculate")
    .addEventListener("click", updateCalculations);
};

const formatValuta = (value) => `${value.toFixed(2).replace(".", ",")} €`;

const updateCalculations = () => {
  let totalSum = 0;

  document.querySelectorAll("table tr").forEach((row, index) => {
    if (index === 0 || row.cells.length < 5) return;

    const priceText = row.cells[1].textContent
      .replace(" €", "")
      .replace(",", ".");
    const price = parseFloat(priceText);
    const amount = parseInt(row.cells[2].querySelector("input").value) || 0;
    const vatText = row.cells[3].textContent.replace("%", "").trim();
    const vat = parseFloat(vatText) / 100;

    const subtotal = amount * price * (1 + vat);
    row.cells[4].textContent = formatValuta(subtotal);
    totalSum += subtotal;
  });

  document.getElementById("totalPrice").textContent = formatValuta(totalSum);
};

window.addEventListener("load", setup);
```

In de JavaScript code hierboven wordt eerst de setup functie aangemaakt. Deze functie zorgt ervoor dat er een EventListener op de knop met ID calculate wordt gezet. Als je op deze knop klikt, wordt de functie updateCalculations uitgevoerd.

De functie formatValuta is een kleine hulpfunctie die een getal omzet naar een nette valuta-weergave, bijvoorbeeld 12.5 wordt "12,50 €".

In de functie updateCalculations wordt eerst een variabele totalSum aangemaakt om de totaalprijs bij te houden. Daarna worden alle rijen van de tabel opgehaald en één voor één doorlopen. De eerste rij (vaak de header) en rijen met te weinig cellen worden overgeslagen.

Voor elke relevante rij wordt de prijs opgehaald uit de tweede cel, de hoeveelheid uit een input in de derde cel en de btw uit de vierde cel. De prijs en btw worden omgezet naar getallen. Vervolgens wordt het subtotaal berekend met de formule: amount * price * (1 + vat). Dit subtotaal wordt in de vijfde cel van de rij gezet, netjes geformatteerd met de formatValuta functie. Ondertussen wordt het subtotaal bij de totalSum opgeteld.

Tot slot wordt de totaalprijs in het element met ID totalPrice gezet, ook weer netjes geformatteerd.